Project Interim Report
On
Online Food Delivery System(Using Core Java & OOP Concepts)
RU-100-01-00012
Object Oriented Programming With Java
SESSION: 2025-26

Submitted By
Keshav K Muley
RU-25-10665

Bachelor of Technology in Computer Science &

Engineering in association with Google

Under the Guidance of

Mr. Soumik Karmakar

IBM SME and EXPERT

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

ABSTRACT

The **Online Food Delivery System** is a Java-based console application designed to handle food ordering transactions across multiple simulated restaurants. Developed in accordance with strict Object-Oriented Programming (OOP) paradigms, the system models real-world business dynamics involving corporate eateries, diverse dynamic menus, custom order selections, and integrated multi-tiered tax and logistic fee estimation algorithms. This application provides automated computational flows for calculating precise billing breakdowns including subtotal aggregates, structural taxation variables, and delivery logic conditions.

This project is needed to demonstrate how real-world commercial food technology networks operate in a simplified, terminal-driven environment. Manual processing of ordering catalogs and structural restaurant pricing dependencies can be highly inefficient and prone to financial sorting errors. Therefore, this framework transitions manual logistics workflows into robust digital structures, minimizing manual data anomalies and maximizing processing consistency. By building encapsulated data layers for discrete system units, this software suite serves as an extensible foundation for modern digital hospitality applications.

INTRODUCTION

The **Online Food Delivery System** is a console-based application developed using Java and object-oriented programming principles. With the rapid growth of digital transformation within food service markets, efficient software suites are required to maintain high throughput across food catalogs, merchant entities, and user order flows. In modern computational systems, transaction automation plays a crucial role in improving logistical reliability, operational precision, and corporate visibility.

This project simulates consumer ordering loops in a structured way, helping developers understand how nested entity variables are instantiated, synchronized, and calculated at runtime. The system allows administrators to establish menus via restaurant profiles, enables users to browse items across indexes, handles variable matching across item counts, and outputs automated tax and delivery charge summaries. By strictly avoiding collections and using native continuous arrays, the application teaches baseline memory management, sequential index traversal optimization, and clear modular method design.

OBJECTIVES

The primary architectural and operational parameters of this structural implementation include:

- To model a real-world multi-restaurant food delivery system.
- To implement core OOP concepts such as encapsulation, classes, objects, and modular methods.
- To manage relational data dependencies between discrete entities like restaurants, food items, and orders.

- To perform accurate, algorithmic financial billing calculations including subtotals, active tax configurations, and delivery incentives.
- To structure system memory using primary arrays rather than utility collections to handle complex dataset traversals.

SCOPE OF THE SYSTEM

This project is designed as an internal backend simulation engine suitable for small-scale local testing, terminal execution environments, and academic evaluations. It functions cleanly as an interactive configuration suite for small local catering environments, restaurant testing hubs, or educational mockups.

In future iterations, the structural limits of this console blueprint can be expanded significantly. The native array allocations can be easily swapped for persistent structural database management engines (such as MySQL or PostgreSQL). Furthermore, the text interface can be modernized by implementing an interactive Graphical User Interface (GUI) via JavaFX or web-based frontend protocols, and connecting real-time geospatial logistics feeds to fetch distance variables dynamically from active network APIs.

SYSTEM REQUIREMENTS

Hardware Requirements:

- **Computing Node:** Workstation/Laptop with a minimum of 4GB RAM (8GB recommended for modern parallel compiler optimization).
- **Processor Infrastructure:** Intel Core i3 or higher / equivalent AMD Ryzen execution core.

Software Requirements:

- **Runtime Toolkit:** Java Development Kit (JDK) Standard Edition version 11 or above.
- **Integrated Development Environment:** Visual Studio Code, Eclipse Enterprise, or IntelliJ IDEA.
- **Operating System Platform:** Microsoft Windows 10/11, Linux Kernel v5.0+, or Apple macOS.

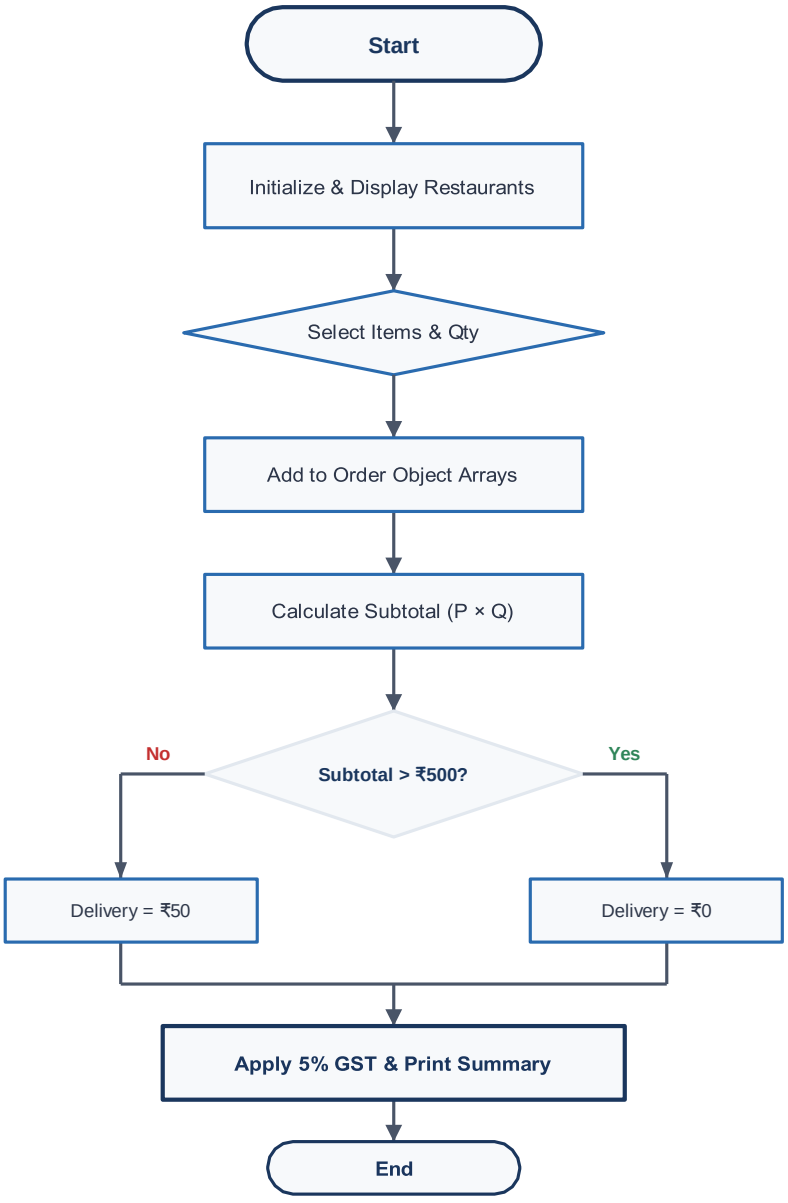
METHODOLOGY

The processing lifecycle and dynamic operational flow of the application execute via sequential control steps:

1. The application initializes, bootstrapping target restaurant objects with predefined multi-item menus.
2. The system prints a terminal-driven workspace context to the console, allowing users to choose operations.
3. The client selects an active restaurant index and views its associated food menu records.
4. The consumer creates a transaction entity, tracking specified items and associated line quantities inside parallel arrays.
5. The application's billing logic automatically calculates subtotals, assesses delivery thresholds, and adds active tax percentages.
6. A compiled transaction receipt block is generated, printing the final itemized statements directly to the console terminal.
7. The runtime interface loops back to the master option menu until an explicit exit call is received.

SYSTEM FLOWCHART

The structural logic, option selection path, and calculation routing implemented across the application workspace follow the flowchart sequence modeled below:


Start
Initialize & Display Restaurants
Select Items & Qty
Add to Order Object Arrays
Calculate Subtotal (P × Q)
Subtotal > ₹500?
No
Yes
Delivery = ₹50
Delivery = ₹0
Apply 5% GST & Print Summary
End

CLASSES, METHODS, & VARIABLES USED

1. Class: FoodItem

Models the individual menu components across restaurant nodes.

- **Variables:**
  - `itemName` (String) — Stores the literal descriptive identifier of the menu item.
  - `price` (double) — Stores the individual unit rate decimal configuration.
- **Methods:**
  - `FoodItem(String name, double price)` — Parametrized initialization constructor mapping instance scope attributes.
  - `getItemName()` — Accessor getter channel outputting name parameters safely.
  - `getPrice()` — Accessor getter channel fetching item baseline rates.

2. Class: Restaurant

Manages specific dining outlets and controls their item menu indices.

- **Variables:**
  - `name` (String) — Defines the physical corporate title of the outlet eatery.
  - `menu` (FoodItem[] Array) — Fixed block index containing instance arrays of individual item dependencies.
- **Methods:**
  - `Restaurant(String name, FoodItem[] menu)` — Assigns specific text identities and bounds corresponding target array components.
  - `displayMenu()` — Traverses structural array blocks to sequentially output item rates and codes.

3. Class: Order

The main engine managing active ordering selections, calculation rules, and transactional receipt compilation.

- **Variables:**
  - `items` (FoodItem[] Array) — Native continuous block holding targeted items selected by users.
  - `quantities` (int[] Array) — Parallel structural array mapping matching quantity integers to ordered items.
  - `subtotal` (double) — Aggregated itemized base total before charges or taxes.
  - `deliveryCharge` (double) — Evaluated logistical transaction fee addition.

- `tax` (double) — Quantified structural percentage mapping applied tax.
- `total` (double) — Final balance aggregate.

- **Methods:**
  - `addItem(FoodItem item, int quantity)` — Appends references to tracking arrays and links quantity weights to the item.
  - `calculateBill()` — Evaluates subtotal sums, checks conditional logistic thresholds, and applies tax calculations.
  - `displaySummary()` — Formats and generates the terminal transaction order breakdown summary.

IMPLEMENTATION DETAILS

The backend engine separates software concerns into three specific domains: `FoodItem`, `Restaurant`, and `Order`. The `FoodItem` component enforces complete object encapsulation by establishing strict variable protection and routing structural field reads exclusively through public getter parameters. The `Restaurant` module serves as a management box for menu segments, using clean primitive array matrices to list target combinations.

The `Order` class acts as the core controller, linking structural data streams without relying on external utilities like `ArrayList`. By mapping tracking selections using parallel arrays, index configurations match perfectly across calculations. Financial flows are completely isolated within clear computational loops, avoiding hardcoded dependencies and using explicit operational rules to determine tax values and delivery exemptions based on transaction totals.

SAMPLE INPUT / OUTPUT

When initialized, the system provides a clean, menu-driven workspace where customers choose paths, review food cards, add items, and receive an itemized receipt summary:

```
========== Online Food Delivery System ==========
1. View Restaurants and Menus
2. Place a New Order
3. Exit
Enter choice: 2

Select Restaurant: 1. Pizza Central  2. Burger Junction
Enter choice: 1

--- Pizza Central Menu ---
1. Margherita Pizza - ₹250
2. Pepperoni Pizza  - ₹350
3. Garlic Bread      - ₹100

Select Item Code to Add: 1
Enter Quantity: 2
Item added successfully.

Do you want to add more items? (yes/no): no

Order Summary:
------------------------
Margherita Pizza x2 = ₹500

Subtotal: ₹500
Delivery Charge: ₹50
Tax (5%): ₹25

Total Amount: ₹575
```

METHODOLOGY & PROJECT TIMELINE

Development progressed through systematic milestones over an estimated 6-day cycle, moving from model architecture creation to final software validation:

Development Stage / Task Identifier	Allocated Duration
Planning & Design (System Layout, Architecture, and UML Flow Maps)	1 Day
Coding & Logic (Model Structures, Enforced Encapsulation, & Array Logic)	2 Days
Billing Optimization (GST Logic loops & Delivery Exception Triggers)	1 Day
System Testing (Terminal Input Validation & Edge-Case Tracing)	1 Day
Technical Documentation (Compiling Final Comprehensive Interim Report)	1 Day
Total Lifecycle Duration	**6 Days**

CONCLUSION

The **Online Food Delivery System** project successfully demonstrates how core Object-Oriented Programming principles can be applied to solve real-world transaction management challenges. By using encapsulated models, clean class separating abstractions, and precise computational methods, the application provides a reliable structure for managing restaurant selections, menus, and customer orders.

Key concepts like native array index tracking, conditional logic, and billing calculation flows were thoroughly verified during development. This project helped enhance overall understanding of primitive dataset management, functional separation parameters, and standard commercial math automation, providing a strong structural baseline for building enterprise-grade applications in the future.